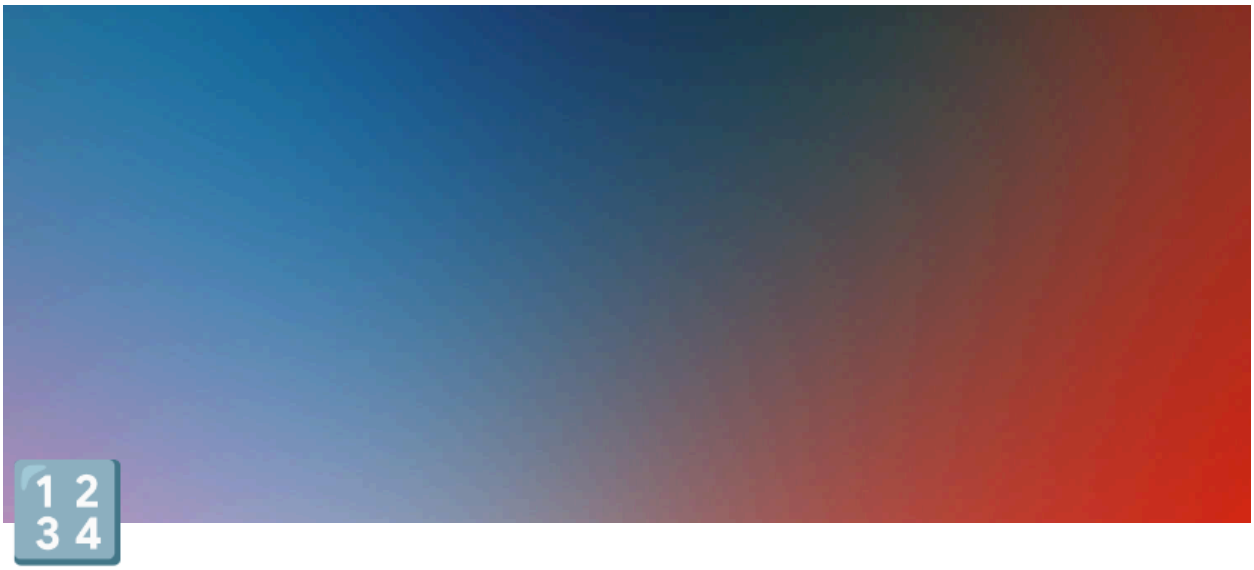


Algoritmos y Estructuras de Datos.

GitHub

Clase 1 (Pasaje de parámetros, Recursión, Listas, Pilas y Colas)

2026_TP1_Listas-1.pdf

Pasaje de parámetros.

En Java los parámetros siempre se pasan por valor, es decir, se copia el valor de una variable cuando se pasa a un método.

- En el primer llamado el resultado es 3.
- En el segundo llamado el resultado es 12.

- En el tercer llamado el resultado es 3.

Esto sucede porque al pasarse el valor 1, se hace una copia del valor del mismo, y no queda reflejado el cambio una vez finalizado el proceso. Esto pasa con todos los tipos primitivos de Java (int, double, long, etc). Aunque utilicemos una clase wrapper, el resultado seria el mismo ya que son inmutables.

```
public class Pruebas {
    public static void main(String[] args) {
        int valor1 = 1;
        int valor2 = 2;

        System.out.println("La suma es: " + (valor1 + valor
2));
        Pruebas.ejemploParametroPrimitivo(valor1,valor2);
        System.out.println("La suma despues de la llamada es:
" + (valor1 + valor2));
    }

    public static void ejemploParametroPrimitivo(int valor1,
int valor2){
        valor1 = 10;
        System.out.println("La suma dentro del metodo es: " +
(valor1 + valor2));
    }
}
```

Tipo de dato referencial.

Si queremos que los datos se vean modificados durante y después de la ejecución del proceso, debemos utilizar un tipo de dato referencial, es decir, pasar la referencia a un objeto, y con ella, cambiar el estado del objeto.

> Dada una lista de números, implementar un método de clase que retorne una nueva lista con únicamente los números pares de la lista original.

```
public static void ejemploDevolverPorParametro(List<Integer>
numeros, List<integer> pares){
     for (int elem : numeros){ // Itero sobre los elementos d
e la lista.
         if (elem % 2 == 0){ // Si el numero es par.
             pares.add(elem); // Agrego a la lista
         }
     }
 }
```

```
List<Integer> numeros = new ArrayList<>(Arrays.asList(1,2,3,
4,5)); // Creo una lista
```

Listas y Recursión.

Una lista es una colección ordenada de elementos, que permite elementos duplicados, se accede mediante un índice y se pueden agregar, eliminar y modificar elementos en cualquier posición. A diferencia de los arreglos (arrays) las listas permiten un tamaño dinámico, es decir, su tamaño puede cambiar durante la ejecución.

```
lista.add(3) // Agregar un elemento.
lista.get(0) // Obtengo el primer elemento.
lista.indexOf(8) // Obtener el indice del elemento 8.
lista.isEmpty() // Funcion que devuelve True si la lista esta
vacia.
lista.remove(0) // Elimino el primer elemento.
lista.size() // Obtengo la dimension logica de la lista.
```

La recursión es una técnica que se usa en informática en la que un método se llama a si mismo para resolver un problema subdividiéndolo en problemas mas pequeños.

> Implementar un método recursivo que cuente cuantos números pares hay en una lista.

```
public static int contarPares(List<Integer> lista){
    if(!lista.isEmpty()){
        return contarParesRecursivo(lista,0);
    }
    else{
        return 0;
    }
}

private static int contarParesRecursivo(List<Integer> lista,
int index){
    if (index == lista.size()){ // Caso base.
        return 0;
    }
    return evaluarPar(lista.get(index)) + contarParesRecursiv
o(lista, index + 1);
}
```

ArrayList: mejor para accesos rápidos.
LinkedList: mejor para modificaciones frecuentes.

Pilas y Colas.

Una pila es una secuencia lineal de objetos actualizada en un extremo llamado tope.
Usa la política LIFO (last in, first out). Se implementa internamente como una lista.

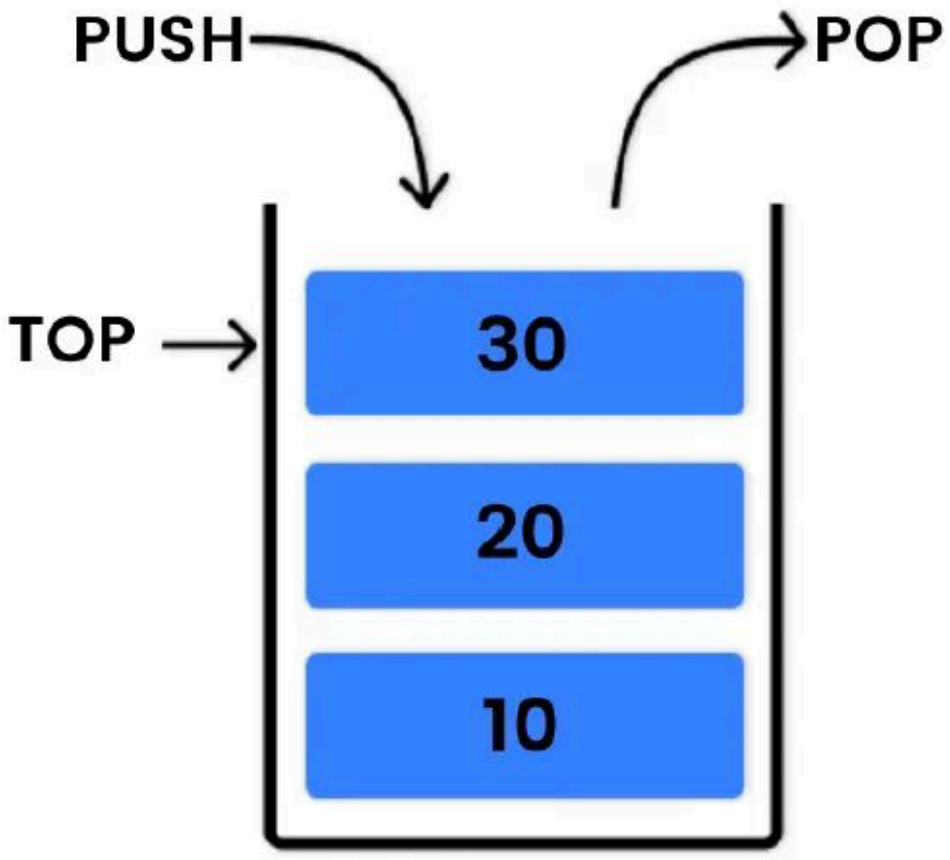
> Algunas operaciones con pilas.

```
Stack<Libro> pila = new Stack<>(); // Se crea una pila de lib
ros.

   pila.push(new Libro("Rayuela", "Julio Cortazar", 1963));
```

```
// Se agrega un elemento.
   pila.push(new Libro("Rayuela", "Julio Cortazar", 1963));
// Se agrega un elemento.
   pila.push(new Libro("Rayuela", "Julio Cortazar", 1963));
// Se agrega un elemento.

   System.out.println(pila); // Muestro todos los elementos d
e la pila.
   System.out.println(pila.top()); // Devuelve el tope de la
pila.
```

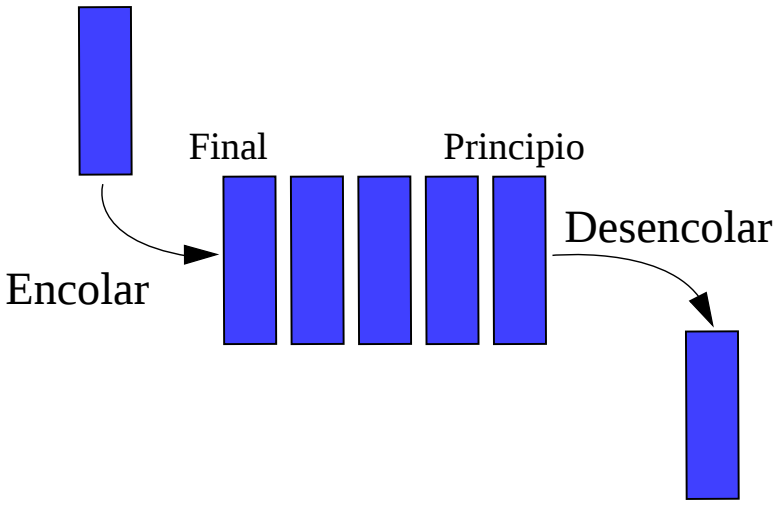


Una cola es una secuencia lineal de objetos actualizada en sus extremos llamados frente y rabo.
Utiliza la politica FIFO (first in, first out, como si fuera una cola del supermercado).
Se implementa internamente con una lista.

> Implementar una simulacion de una caja de supermercado donde los clientes tiene una cantidad de productos y se calcula el tiempo total de atencion. Cada cliente tiene su nombre y la cantidad de productos que compra. El cajero tarda 2 segundos por cada producto procesado.

```
public static int simulacion(Queue<Clientes>clientes){
     int tiempoTotal = 0;
     while (!clientes.isEmpty()){
         Cliente atendido = clientes.dequeue();
         int tiempo = atendido.getCantProductos() * 2;
         tiempoTotal += tiempo;
     }
     return tiempoTotal;
}
public static void main(String[] args) {
     Queue<Cliente> clientes = new Queue();

     clientes.enqueue(new Cliente("Juan", 5)); // Se encola u
n cliente
     clientes.enqueue(new Cliente("Marcos", 4)); // Se encola
un cliente
     clientes.enqueue(new Cliente("Pedro", 1)); // Se encola
un cliente
}
```



Clase 2 (Arboles Binarios)

2026_TP2_AB.pdf

Un árbol binario es una estructura de datos en la que cada nodo puede tener **a lo sumo 2 hijos**: izquierdo y derecho. Cada nodo almacena un dato y referencias a sus hijos, lo que permite modelar relaciones jerárquicas. En esta primera aproximación vamos a tratar el árbol **sin un criterio de orden** (a diferencia de un ABB), por lo que la “posición” de los elementos depende de cómo lo construyamos.
Esta es la clase que nos provee la catedra para trabajar con arboles binarios:

```
public class BinaryTree <T>{
    private T data;
    private BinaryTree<T> leftChild;
    private BinaryTree<T> rightChild;
}
```

Conceptos útiles:
- **Raíz**: nodo inicial del árbol.
- **Subárbol**: un nodo con todos sus descendientes (por ejemplo, el subárbol

izquierdo).
- **Hoja**: nodo sin hijos.
- **Altura / nivel**: medidas que se suelen usar para analizar el árbol (dependen de la cátedra / definición usada).

Algunos métodos que posee la clase BinaryTree son:

`getRightChild()`	Devuelve el hijo derecho.
`getLeftChild()`	Devuelve el hijo izquierdo.
`addLeftChild()`	Agrega al hijo izquierdo.
`addRightChild()`	Agrega al hijo derecho.
`removeLeftChild()`	Elimina el hijo izquierdo.
`removeRightChild()`	Elimina el hijo derecho.
`isEmpty()`	Devuelve true si el árbol esta vacío.
`isLeaf()`	Devuelve true si es una hoja.
`hasLeftChild()`	Devuelve true si tiene hijo izquierdo.
`hasRightChild()`	Devuelve true si tiene hijo derecho.

Estos métodos se usan mucho para **validar casos borde** (por ejemplo, antes de acceder a un hijo) y para escribir recorridos sin errores.

Setup basico de un arbol binario en Java.

```
    BinaryTree<Integer> ab = new BinaryTree<Integer>(40);
    BinaryTree<Integer> hijoIzquierdo = new BinaryTree<Intege
r>(25);
    hijoIzquierdo.addLeftChild(new BinaryTree<Integer>(10));
    hijoIzquierdo.addRightChild(new BinaryTree<Integer>(32));
    BinaryTree<Integer> hijoDerecho = new BinaryTree<Integer>
(78);
    ab.addLeftChild(hijoIzquierdo);
    ab.addRightChild(hijoDerecho);
}
```

En este ejemplo, `ab` es la raíz (40). Su hijo izquierdo es 25 (con hijos 10 y 32) y su hijo derecho es 78.

```
public void printPreOrden(){
    System.out.println(this.getData() + "///");
    if (this.hasLeftChild()){
        this.getLeftChild().printPreOrden();
    }
    if (this.hasRightChild()){
        this.getRightChild().printPreOrden();
    }
}
```

Este método imprime un **recorrido preorden** (visito el nodo actual y luego recorro el subárbol izquierdo y el derecho). Es un ejemplo típico de cómo se aprovecha la recursión en árboles.

Hay dos grandes familias dentro de los recorridos de los arboles binarios:
- **Recorrido en profundidad**: utiliza recursión para recorrer el arbol (por ejemplo, preorden, inorden y postorden).
- **Recorrido por niveles**: utiliza iteración para recorrer el árbol, normalmente con una cola (BFS).

> Escribir en una clase "EjercitacionArboles" el metodo publico con la siguiente firma:
> `public BinaryTree<Integer>sumarArboles(BinaryTree<Integer> arbol1, BinaryTree<Integer> arbol2)` .
> Si ambos arboles tienen la misma estructura, el metodo:
> - Crea y devuelve un nuevo arbol.
> - Donde cada nodo del nuevo árbol contiene la suma de los nodos correspondientes entre el árbol 1 y 2.
> Si no tienen la misma estructura devuelve un árbol vacío.

```
public class EjercitacionArboles {
    public BinaryTree<Integer> sumarArboles (BinaryTree<Integ
er>arbol1, BinaryTree<Integer>arbol2){
```

```
        BinaryTree<Integer> arbolSuma = new BinaryTree<Intege
r>();

        if (arbol1 == null || arbol2 == null) return null;
        if (arbol1.isEmpty() || arbol2.isEmpty()) return arbo
lSuma;

        boolean estructuraValida = sumaRecursiva(arbol1, arbo
l2, arbolSuma);
        return estructuraValida ? arbolSuma : new BinaryTree<
Integer>();

        private static boolean sumaRecursiva(BinaryTree<Integ
er>arbol1, BinaryTree<Integer>arbol2, BinaryTree<Integer>arbo
lSuma){
            boolean estructuraValida = true;

            arbolSuma.setData(arbol1.getData() + arbol2.getDa
ta());
            if (arbol1.hasLeftChild() && arbol2.hasLeftChild
()){
                // Testeo para no llamar con las partes izqui
erdas si no coinciden en estructuras derechas.
                if (arbol1.hasRightChild() == arbol2.hasRight
Child){
                    arbolSuma.addLeftChild(new BinaryTree<Int
eger>());
                    estructuraValida = sumaRecursiva(arbol1.g
etLeftChild(), arbol2.getLeftChild(), arbolSuma.getLeftChild
());
                }
                else estructuraValida = false;
            }
            else{
                estructuraValida = arbol1.hasLeftChild() == a
rbol2.hasLeftChild();
```

```
            }
            if (estructuraValida)
                if (arbol1.hasRighChild() && arbol2.hasRighCh
ild()){
                    arbolSuma.addRightChild (new BinaryTree<I
nteger>());
                    estructuraValida = sumaRecursiva(arbol1.g
etRightChild(), arbol2.getRightChild(), arbolSuma.getRightChi
ld());
                }
                else {
                    estructuraValida = arbol1.hasRighChild()
== arbol2.hasRighChild();
                }
        }   return estructuraValida;
    }
}
```

Recorridos de Arboles Binarios

Preorden (preorder): Es un recorrido en profundidad en el cual se **visita primero la raíz del árbol**, y luego se recorren recursivamente el **subárbol izquierdo** y el **subárbol derecho**.
👉 Raíz → Subárbol izquierdo → Subárbol derecho.

```
public void preOrden() {
    System.out.print(this.data + " "); // raíz

    if (this.leftChild != null) {
        this.leftChild.preOrden(); // izquierdo
    }

    if (this.rightChild != null) {
        this.rightChild.preOrden(); // derecho
```

```
    }
}
```

Inorden (inorder): es un recorrido en profundidad en el cual se recorre primero el subárbol izquierdo, luego se visita la raíz y finalmente se recorre el subárbol derecho.
👉 Subárbol izquierdo → Raíz → Subárbol derecho.

```
public void inOrden() {
    if (this.leftChild != null) {
        this.leftChild.inOrden(); // izquierdo
    }

    System.out.print(this.data + " "); // raíz

    if (this.rightChild != null) {
        this.rightChild.inOrden(); // derecho
    }
}
```

Postorden (postorder): es un recorrido en profundidad en el cual se recorren primero los subarboles izquierdo y derecho y recien al final se visita la raiz.
👉 Subárbol izquierdo → Subárbol derecho → Raíz.

```
public void postOrden() {
    if (this.leftChild != null) {
        this.leftChild.postOrden(); // izquierdo
    }

    if (this.rightChild != null) {
        this.rightChild.postOrden(); // derecho
    }
```

```
    System.out.print(this.data + " "); // raíz
}
```

Todos estos recorridos son de tipo **DFS (Depth-First Search / búsqueda en profundidad)**, ya que exploran completamente un subárbol antes de pasar al siguiente.

La diferencia entre ellos está únicamente en **el momento en que se procesa la raíz** respecto a sus subárboles.